CMMI (CMM Integrated)

- CMMI is the successor of the CMM.
- In 2002, CMMI Version 1.1 was released.
  - Version 1.2 followed in August 2006.
- **The goal of CMMI is to integrate many different models into one framework.**

Three Flavours of CMMI

- **CMMI for Development**
  - Encompasses software engineering, systems engineering, collaborative teams, acquisition from the suppliers.
- **CMMI for Services**
  - Organizations that provide services (from taxi aggregators to call centers)
- **CMMI for Acquisition**
  - Acquiring products and services

Personal Software Process (PSP)

- Based on the work of Humphrey.
- PSP is a scaled down version of industrial software process:
  - Suitable for individual use.
- Even CMM assumes that engineers use effective personal practices.

Personal Software Process (PSP)

- A process is a set of steps for doing a job.
- The quality and productivity of an engineer
  - **Largely determined by his process**
- PSP framework:
  - **Helps a software engineer to measure and improve the way he works.**

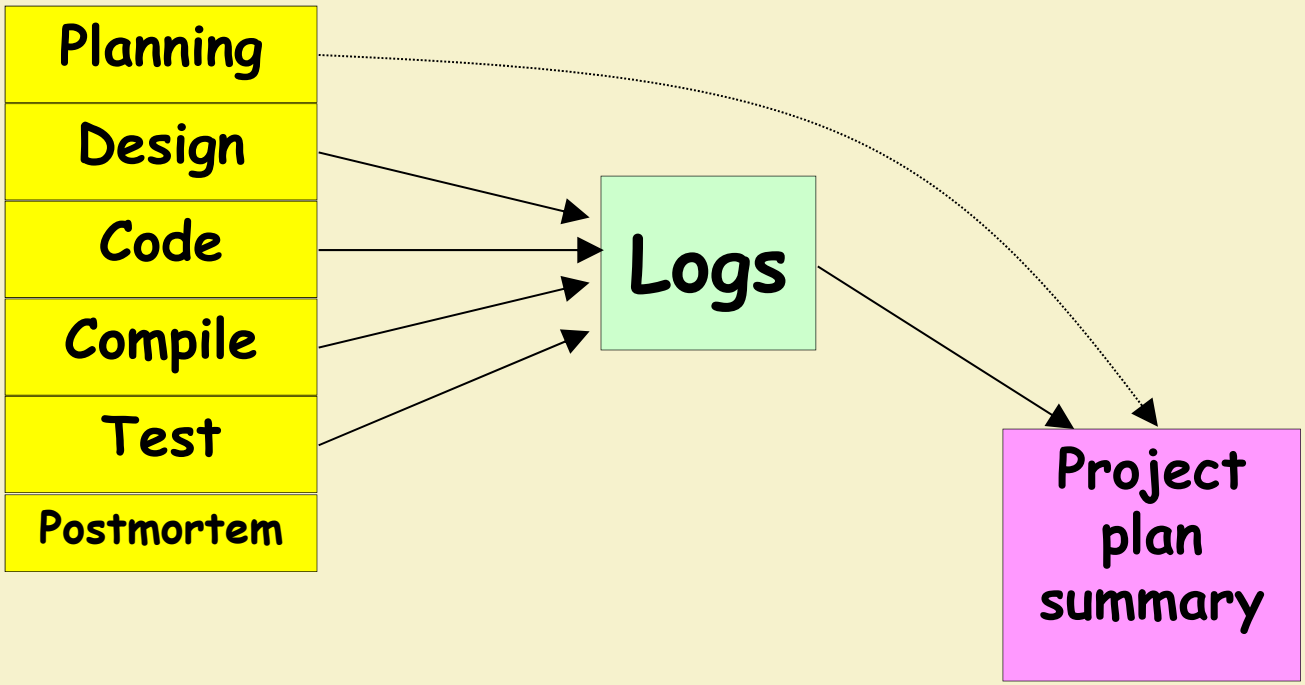
Personal Software Process (PSP)

- **Recognizes that a process for individual use is different from that for a team project.**
  - Provides a defined process:
    - Can be fine tuned by individuals
- Helps developing personal skills and methods.
  - Estimating and planning.
  - Shows how to track performance against plans.

Time Management

- Track the way you spend time:
  - Boring activities seem longer than actual.
  - Interesting activities seem short.
- Record time for:
  - Designing
  - Writing code
  - Compiling
  - Testing

Personal Software Process (PSP)



PSP-Planning

- Problem definition
- Estimate max, min, and total LOC
- Determine minutes/LOC
- Calculate max,min, and total development times
- Enter the plan data in project plan summary form
- Record the planned time in Log

PSP-Design

- Design the program.
- Record the design in specified format.
- Record the Design time in time recording log.

PSP-Code

- Implement the design.
- Use a standard format for code text.
- Record the coding time in time recording log.

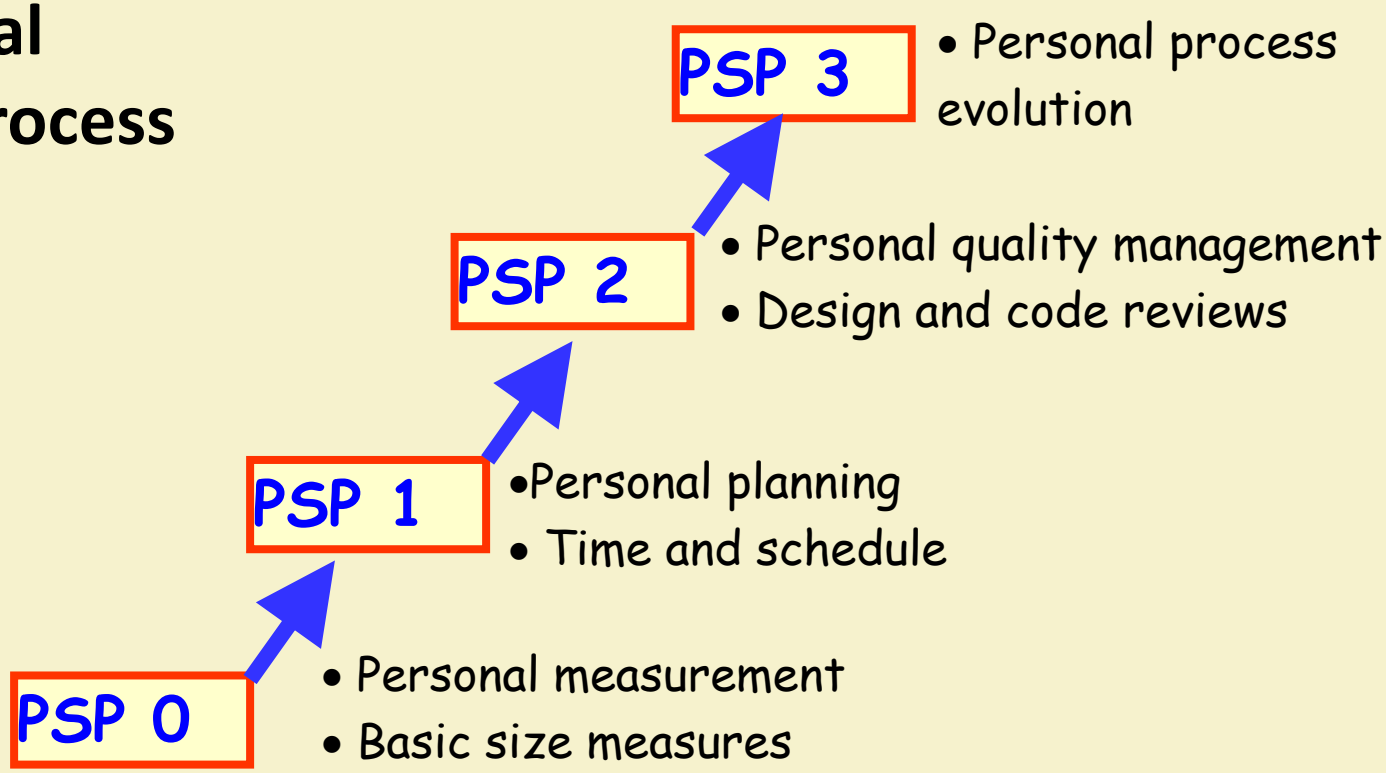
PSP-Compile

- Compile the program.
- Fix all the defects.
- Record compile time in time recording log.

PSP-Test/Post mortem

- **Test:**
  - Test the program.
  - Fix all the defects found.
  - Record testing time in time recording log.
- **Postmortem:**
  - Complete project plan summary form with actual time and size data.
  - Record post mortem time in time record.

Personal Software Process (PSP)



Six Sigma

- Six sigma is a quantitative approach to eliminate defects:
  - Applicable to all types of industry - from manufacturing, product development, to service.
- The statistical representation of Six Sigma quantitatively describes :
  - How a process is performing.

6 σ (6 sigma)

- A defect is anything that does not fall within the customer's tolerance limits
- Through *continuous process improvement*,
  - **Lower the process variability so that the upper and lower limits are 6 standard deviations above and below the mean**

Achieving Six Sigma

- To achieve six sigma:
  - **A process must not produce more than 3.4 defects per million opportunities.**
  - 5 Sigma -> 230 defects per million.
  - 4 Sigma -> 6210 defects per million.
- Pareto Chart:
  - Simple bar chart to represent defect data
  - Used to identify the problems that occurs with greatest frequency
    - or incur the highest cost

Six Sigma Methodologies

- Six sigma methodologies:
  - DMAIC (Define, Measure, Analyze, Improve, Control).
  - DMADV: (Define, Measure, Analyze, Design, Verify).
- The methodologies are implemented by Green belt and Black belt workers:
  - Supervised by Master black belt worker.

NPTEL ONLINE CERTIFICATION COURSES



Course Name: Software Project Management

Faculty Name: RAJIB MALL

Department : Computer Science and Engineering

TOPIC: Software Reliability



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Introduction

- Reliability of software:
  - A concern of most users --- especially the industry users.
  - An important determinant of the quality of the software.
- Users not only want highly reliable software:
  - **Want quantitative estimation of reliability before making buying decision.**

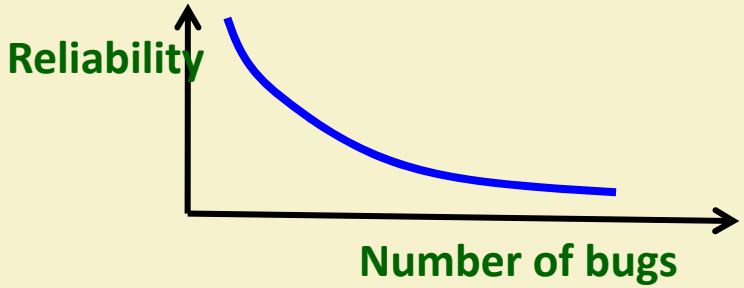
Software Reliability: Definitions

- Informally denotes trustworthiness or dependability of software.

- **Probability of the software working "correctly" over a given period of time under specified operating conditions. IEEE 1991**

Software Reliability

- Intuitively:
  - A software having a large number of defects is unreliable.
- It is also clear:
  - Reliability of a software improves if the number of defects is reduced.



Software Reliability Measurement

- Reliability evaluation of a software:
  - **Essentially involves assigning a number to it to indicate its propensity to fail.**
- Accurate software reliability evaluation:
  - A very difficult problem... Why?
  - **Several factors make software reliability measurement difficult...**

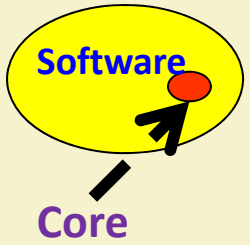
CONCEPTS COVERED

- Software Reliability Measurement
- Reliability Growth Modelling
- Statistical testing



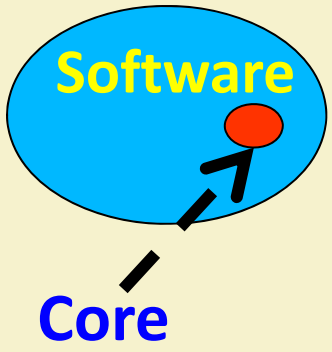
Major Problems in Reliability Measurements (1)

- All errors do not cause failures at the same frequency and severity.
  - Removing errors from parts of software which are rarely used: Makes little difference to the perceived reliability.



→**Estimating latent errors alone not enough to determine reliability.**

The 90-10 Rule



- Experiments from analysis of behavior of a large number of programs:
  - **90% of the total execution time is spent in executing only 10% of the instructions in the program.**
- The most used 10% instructions:
  - called the core of the program.

Effect of 90-10 Rule on Software Reliability

- Least used 90% statements:
  - called non-core are executed only during 10% of the total execution time.
- It may not be very surprising then:
  - Removing 60% defects from least used parts leads to only about 3% improvement to product reliability.

Difficulty in Software Reliability Measurement

- Reliability improvements from correction of a single error:
  - **Depends to a large extent on whether the error belongs to the core or the non-core part of the program.**

Difficulties in Software Reliability Measurement (2)

- No simple relationship exists between:
  - Observed system reliability, and
  - Number of latent software defects.
- **Failure rate is observer-dependent!**

Reliability is Observer-Dependent

- If we select test input, such that:
  - only "correctly" implemented functions are executed,
    - None of the errors will be exposed
    - Perceived reliability of the product will be high.

Reliability is Observer-Dependent

- If we select input data, such that :
  - Only functions containing many errors are invoked.
  - Perceived reliability of the system will be low.

Software Reliability

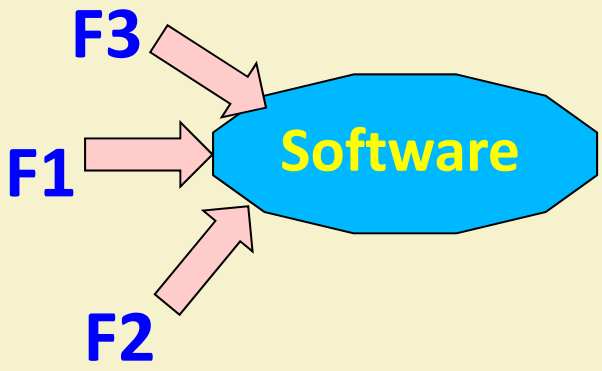
- Different users use a software application in different ways.
  - Failures which show up for one user,
    - may not show up for another.
- Reliability of a software product:
  - **Clearly observer-dependent**
  - **Difficult to determine absolutely.**

Operational Profile

- The perceived reliability depends to a large extent upon:
  - How the software is used...
  - In technical terms on its **operational profile.**

Operational Profile

Function	Probability of occurrence
F1:f11	0.4
f12	0.1
f13	0.1
F2:f21	0.05
f22	0.15
F3:f31	0.15
f32	0.05



Difficulty in Software Reliability Measurement (3)

- **Software reliability keeps changing throughout the life of the product**
  - Each time an error is detected and corrected, the reliability changes.
  - By how much does the reliability increase when a failure is detected and bug is fixed?

Hardware vs. Software Reliability



- Hardware failures:
  - Inherently different from software failures.
- Most hardware failures are due to component wear and tear:
  - Some component no longer functions as specified.

Hardware vs. Software Reliability

- A logic gate can be stuck at 1 or 0,
  - –or a resistor might short circuit.
- To fix hardware faults:
  - –Replace or repair the failed part.

Hardware vs. Software Reliability

- Software failures are never due to wear and tear!

  ➔ system will continue to fail:

  - Unless changes are made to the software design and code.

Hardware vs. Software Reliability

- Many metrics exist for satisfactory measurement of hardware reliability.
- These however are not very meaningful for software reliability measurements:
  - **Because of the fundamental differences between their characteristics.**

Difficulty in Software Reliability Measurement

- Whenever a software fails a test case, it is debugged and the fault is rectified.
  - **The traditional notion of measuring reliability by observing the number of failures per unit time therefore does not appear to be meaningful in the software context.**

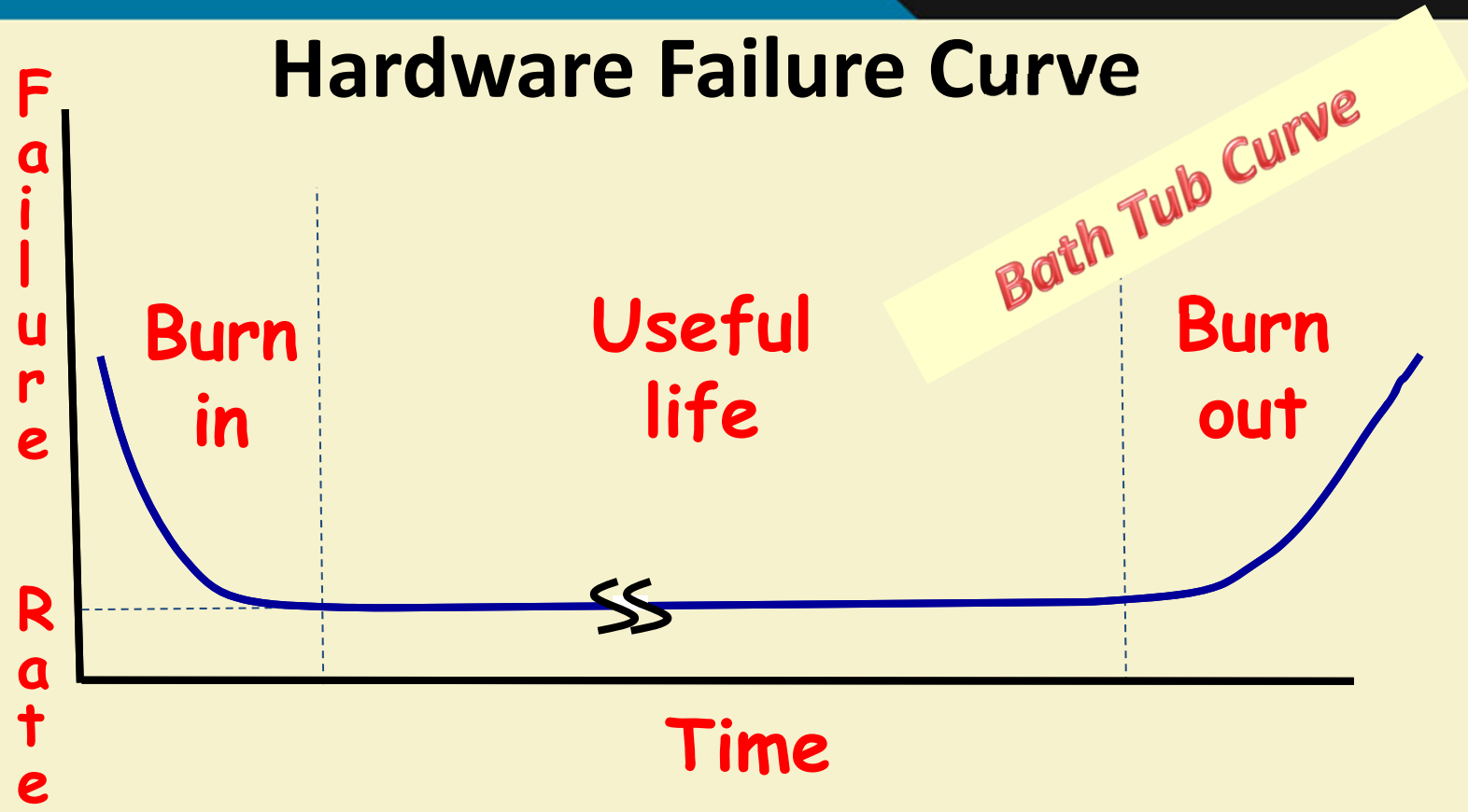
Hardware vs. Software Reliability

- When a hardware system is repaired:
  - Its reliability is maintained
- When software is repaired:
  - Its reliability may increase or decrease.

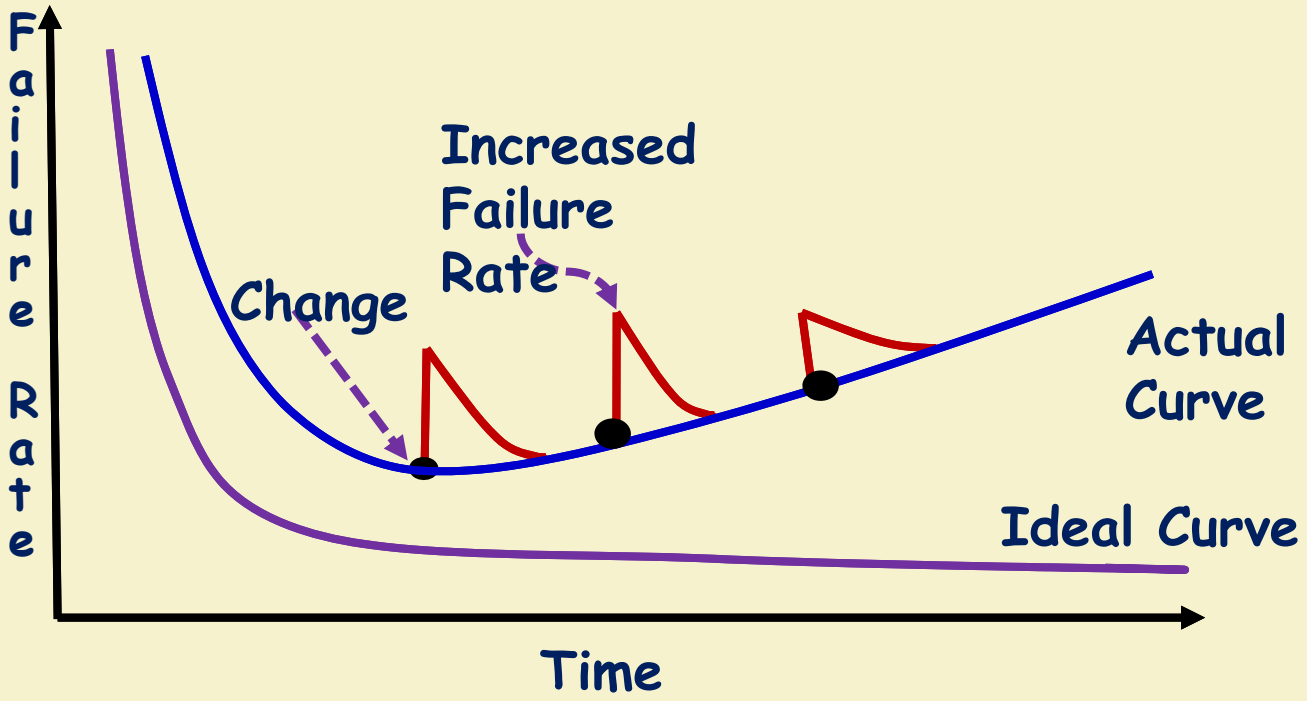
Hardware vs. Software Reliability

- Goal of hardware reliability study:
  - Stability (i.e. interfailure times remains constant)
- Goal of software reliability study
  - Reliability growth (i.e. interfailure times increases)

Hardware Failure Curve



Software Failure Curve



Measuring Reliability: Metrics

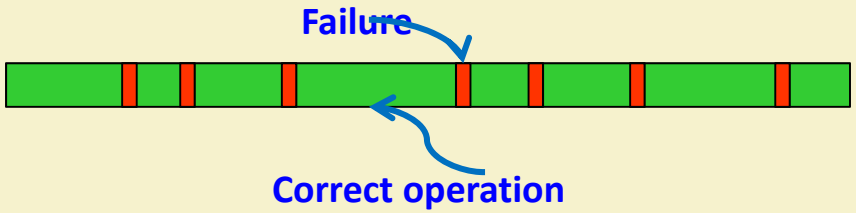
- Different categories of software products have different reliability requirements:
  - **Level of reliability required for a software should be specified in the SRS document.**

Reliability Metrics

- A good reliability measure should be observer-independent,
  - So that different people can agree on the reliability.

Mean Time To Failure (MTTF)

- Average time between two successive failures:
  - Observed over a large number of failures.



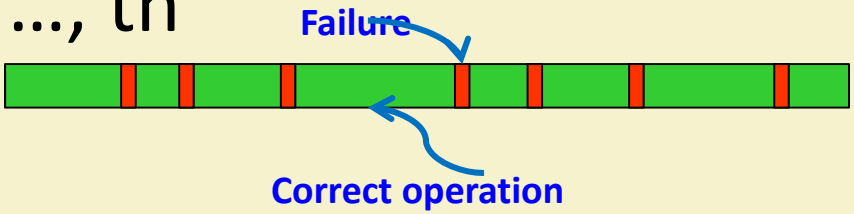
Mean Time To Failure (MTTF)

- **MTTF is not as appropriate for software as for hardware:**
  - A Hardware system fails due to a component's wear and tear.
    - MTTF indicates how frequently the system fails
  - When a software error is repaired:
    - the same error never appears.

Mean Time To Failure (MTTF) Measurement

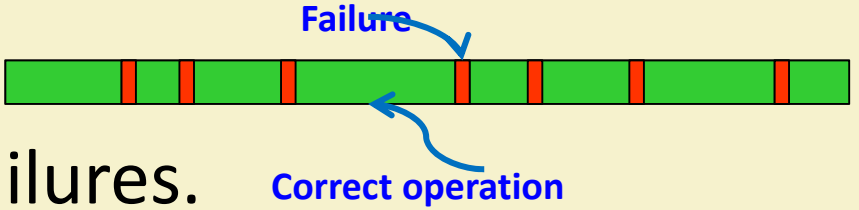
- Record failure data for n failures:
  - let these be t1, t2, …, tn
  - calculate $(t_{i+1}-t_i)$
  - the average value is MTTF

$$\sum(t_{i+1}-t_i)/(n-1)$$



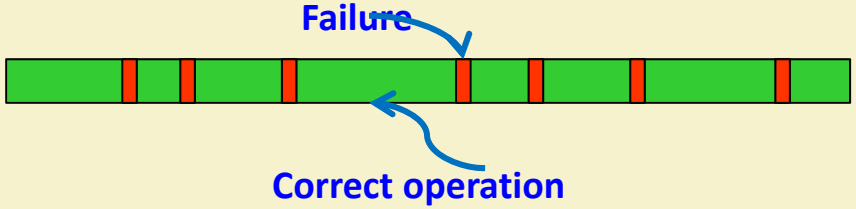
Rate of occurrence of failure (ROCOF)

- ROCOF measures:
  - Frequency of occurrence failures.
  - Observe the behavior of a software product while in operation:
    - over a specified time interval
    - calculate the total number of failures during the interval.



Mean Time to Repair (MTTR)

- Once failure occurs:
  - Some time taken to fix faults



- MTTR:
  - Measures average time it takes to fix faults.

Mean Time Between Failures (MTBF)

- We can combine MTTF and MTTR:
  - to get an availability metric:
  - **MTBF=MTTF+MTTR**
- MTBF of 100 hours would indicate:
  - Once a failure occurs, the next failure is expected after 100 hours of clock time (not running time).

Probability of Failure on Demand (POFOD)

- Unlike other metrics
  - This metric does not explicitly involve time.
- Measures the likelihood of the system failing when a service request is made:
  - POFOD of 0.001 means:
    - 1 out of 1000 service requests may result in a failure.

Availability

- Measures the likelihood of the of the system being available for use over a period of time:
  - Considers the number of failures occurring during a time interval,
  - Also takes into account the repair time (down time) of a system.

Availability

- This metric is important for systems which are supposed to be never down:
  - Where loss of service during repair and restart is important.
- Examples:
  - telecommunication systems,
  - operating systems, etc.

Availability

- Operational availability

$$A_O = \frac{MTBF}{MTBM + MDT}$$

- Inherent availability

$$A_O = \frac{MTBF}{MTBF + MTTR}$$

MTBM = mean time between maintenance

MDT = mean down time

MTBF = mean time between failures

MTTR = mean time to repair

Reliability metrics

- Almost every reliability metric:
  - centers around the probability of system failures.
  - Takes no account of the consequences of failures.
    - Severity of failures may be very different.

Reliability metrics

- Failures which are transient and whose consequences are not serious:
  - Of little practical importance in the use of a software product.
  - Such failures can at best be minor irritants.

Failure Classes

- More severe types of failures:
  - may render the system totally unusable.
- To accurately estimate reliability of a software product:
  - it is necessary to classify different types of failures.

Failure Classes

- **Transient**:
  - Transient failures occur only for certain inputs.
- **Permanent**:
  - Permanent failures occur for all input values.
- **Recoverable**:
  - When recoverable failures occur:
    - System recovers with or without operator intervention.

Failure Classes

- **Unrecoverable**:
  - the system may have to be restarted.
- **Cosmetic**:
  - These failures just cause minor irritations,
    - do not lead to incorrect results.
  - An example of a cosmetic failure:
    - mouse button has to be clicked twice instead of once to invoke a GUI function.

CONCEPTS COVERED

- Software Reliability Estimation
- Reliability Growth Modeling
- Statistical Testing



How to Estimate The Reliability of a Software Application?

- Reliability Growth Modelling
- Statistical testing
- Indirect means

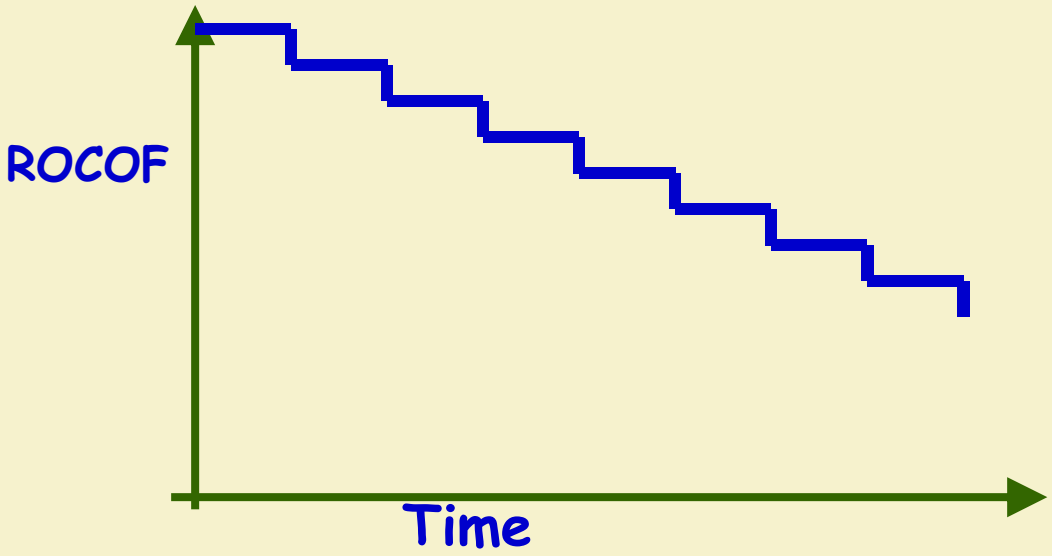
Reliability Growth Modelling

- A reliability growth model:
  - **Models how software reliability grows with time**
    - **as errors are detected and repaired.**
- A reliability growth model can be used to predict:
  - When (or if at all) a particular level of reliability is likely to be attained.
  - How long to test the system?

Step Function Model

- The simplest reliability growth model:
  - a step function model
- The basic assumption:
  - **Reliability increases by a constant amount each time an error is detected and repaired.**

Step Function Model



Step Function Model

- Assumes:
  - All errors contribute equally to reliability growth
  - Unrealistic:
    - We already know that different errors contribute differently to reliability growth.

Jelinski and Moranda Model

- Assumes:
  - Failure rate is proportional to the number of faults remaining in the software
  - Each time an error is repaired reliability does not increase by a constant amount.
- Reliability improvement due to fixing of an error:
  - **Assumed to be a function of the number of errors present in the system at that time.**

Jelinski and Moranda Model

- Realistic for many applications,
  - Still suffers from several shortcomings.
  - **Most probable failures (failure types which occur frequently):**
    - **Discovered early during the testing process.**

Jelinski and Moranda Model

- Repairing faults discovered early:
  - Should contribute maximum to the reliability growth.
- **Rate of reliability growth should be large initially:**
  - **Slow down later on,**
  - Contrary to assumption of the model

Littlewood and Verall's Model

- Allows for negative reliability growth:
  - When software repair introduces further errors.
  - Models the fact that as errors are repaired:
    - Average improvement in reliability per repair decreases.

Littlewood and Verall's Model

- Treats a corrected bug's contribution to reliability improvement:
  - An independent random variable having Gamma distribution.
- Removes bugs with large contributions to reliability:
  - Earlier than bugs with smaller contribution
  - Represents diminishing return as test continues.

Reliability growth models

- There are more complex reliability growth models,
  - More accurate approximations to the reliability growth.
  - These models are out of scope of our discussion.

Applicability of Reliability Growth Models

- There is no universally applicable reliability growth model.
  - Reliability growth is not independent of application.
- **Fit observed data to several growth models.**
  - **Take the one that best fits the data.**

Statistical Testing

- A testing process:
  - **The objective is to determine reliability rather than discover errors.**
  - Uses data different from defect testing.
  - Test data defined based on the operational profile

How to define operational profile?

- **Divide the input data into a number of input classes:**
  - e.g. create, edit, print, file operations, etc.
- **Assign a probability value to each input class:**
  - Based on the probability that an input value from that class to be selected for a test case.

Steps involved in Statistical testing (Step-I)

- Determine the operational profile of the software:
  - This can be determined by analyzing the usage pattern.

Function	Probability
F1:f11	0.4
f12	0.1
f13	0.1
F2:f21	0.05
f22	0.15
F3:f31	0.15
f32	0.05

Statistical testing: Step 2

- Manually select or automatically generate a set of test data:
  - Corresponding to the operational profile.

Function	Probability
F1:f11	0.4
f12	0.1
f13	0.1
F2:f21	0.05
f22	0.15
F3:f31	0.15
f32	0.05

Statistical testing: Step 3

- Apply test cases to the program:
  - Record execution time between each failure
  - It may not be appropriate to use elapsed time

Statistical testing: Step 4

- After a statistically significant number of failures have been observed:
  - Reliability can be computed.

Statistical Testing

- Relies on using large test data set (statistically significant).
- Assumes that only a small percentage of test inputs:
  - likely to cause system failure.

Statistical Testing

- It is straight forward to generate tests corresponding to the most common inputs:
  - **but a statistically significant percentage of unlikely inputs should also be included.**
- Creating these may be difficult:
  - especially if test generators are used.

Advantages of Statistical Testing

- Concentrate on testing parts of the system most likely to be used:
  - results in a system that the users find more reliable (than actually it is!).

Advantages of Statistical Testing

- Reliability predictions based on test results:
  - **Gives an accurate estimation of reliability (as perceived by the average user) compared to other types of measurements.**

Disadvantages of Statistical Testing

- It is not easy to do statistical testing properly:
  - **There is no simple or repeatable way to accurately define operational profiles.**
  - **Selecting proportionate unlikely inputs (corner cases)**

Summary

- Reliability of a software product:
  - Essentially denotes its trustworthiness or dependability.
  - Probability of the product working "correctly" over a given period of time.

Summary

- Operational profile of a software
  - Reflects how it will be used in practice.
  - Consists of specification of:
    - classes of inputs
    - probability of their occurrence.

Summary

- Statistical testing:
  - **Based on testing using large data set selected as per operational profile.**
  - Provides realistic reliability figures.

NPTEL ONLINE CERTIFICATION COURSES



Course Name: Software Project Management

Faculty Name: RAJIB MALL

Department : Computer Science and Engineering

TOPIC: Software Testing



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Faults and Failures

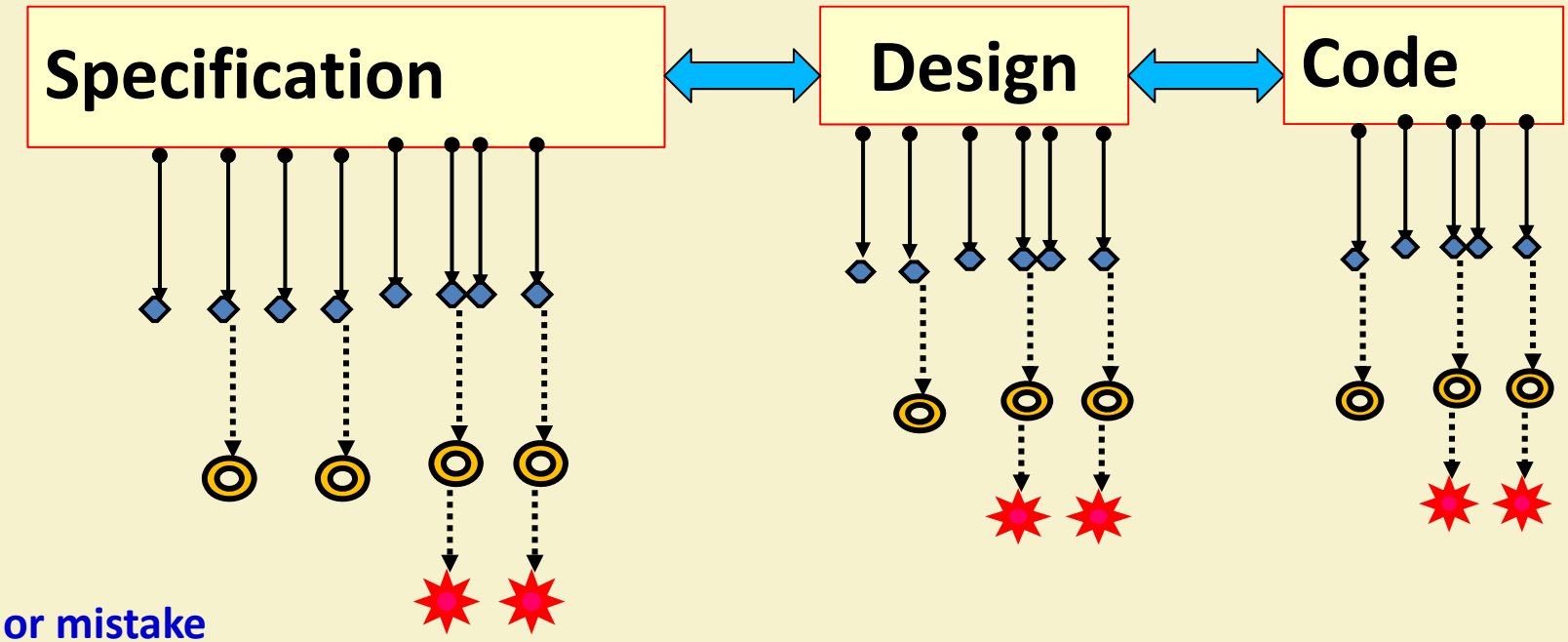


- A program may fail during testing:
  - Essentially a manifestation of a fault (also called defect or bug).
  - **Mere presence of a fault may not lead to a failure.**

Errors, Faults, Failures

- Programming is human effort-intensive:
  - Therefore, inherently error prone.
- IEEE std 1044, 1993 defined errors and faults as synonyms :
- **IEEE Revision of std 1044 in 2010 introduced finer distinctions:**
  - For more expressive communications distinguished between Errors and Faults





Error or mistake

Fault, defect, or bug

Failure

Error Tit-Bits

- Even experienced programmers make many errors:
  - Avg. 50 bugs per 1000 lines of source code
- Extensively tested software contains:
  - **About 1 bug per 1000 lines of source code.**
- Bug distribution:
  - 60% spec/design, 40% implementation.



Bug Source

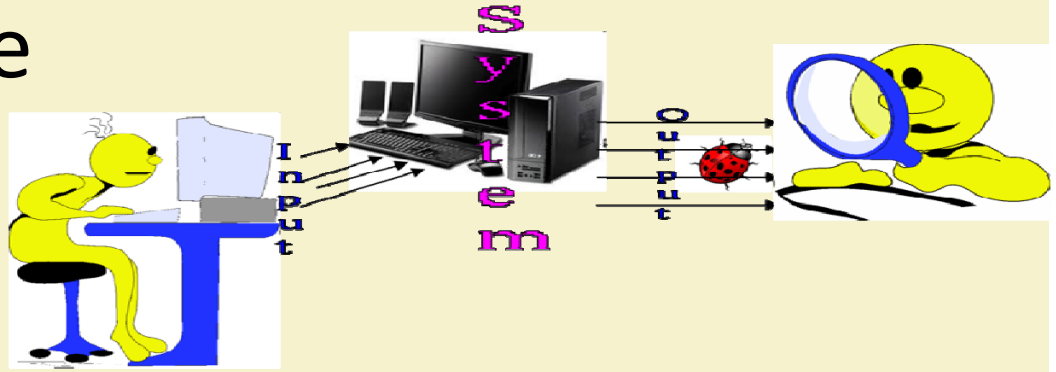


How are Bugs Reduced?

- Review
- **Testing**
- Formal specification and verification
- Use of development process

How to Test?

- Input test data to the program.
- Observe the output:



  - Check if the program behaved as expected.

Examine Test Result…

- If the program does not behave as expected:
  - Note the conditions under which it failed (Test report).
  - Later debug and correct.

Testing Facts

- Consumes the largest effort among all development activities:
  - Largest manpower among all roles
  - Implies more job opportunities
- About 50% development effort
  - But 10% of development time?
  - How?

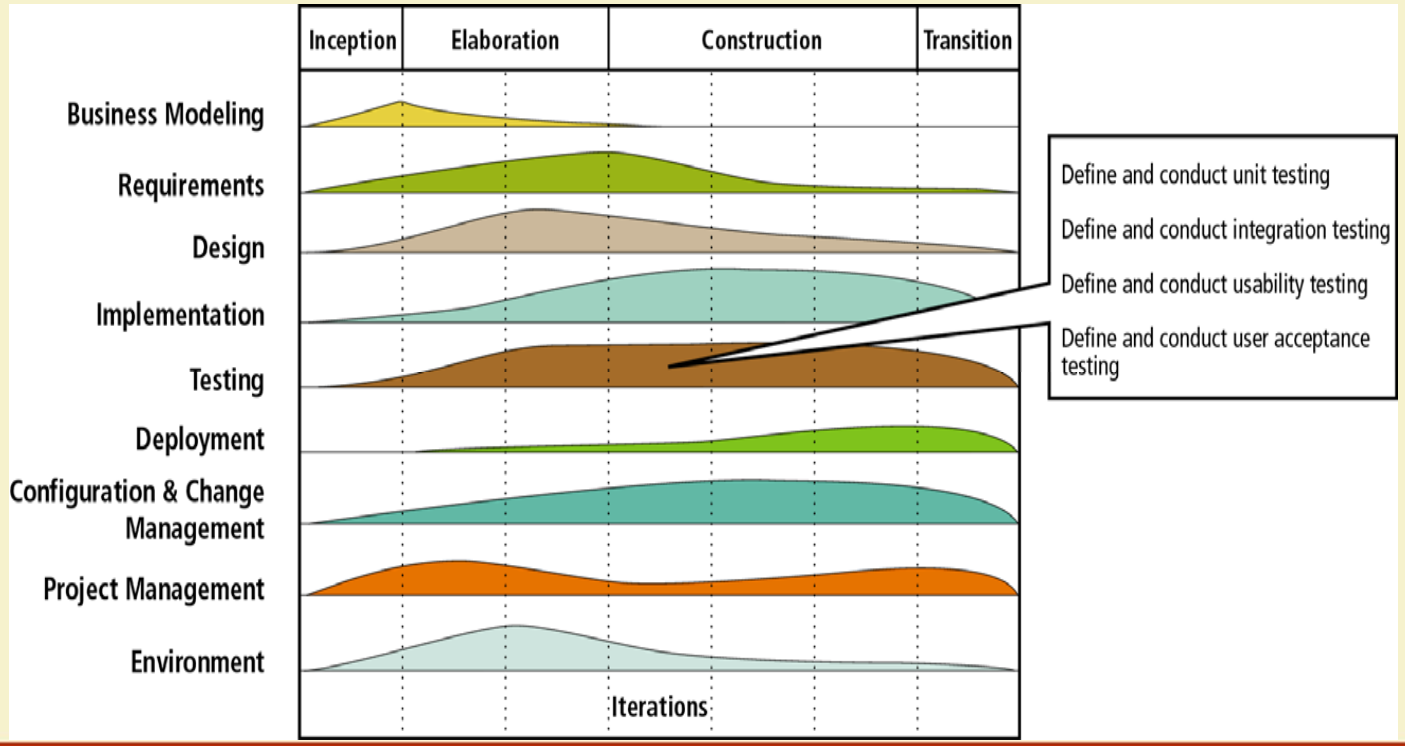
Testing Facts

- Testing is getting more complex and sophisticated every year.
  - **Larger and more complex programs**
  - **Newer programming paradigms**
  - **Newer testing techniques**
  - **Test automation**

Testing Perception

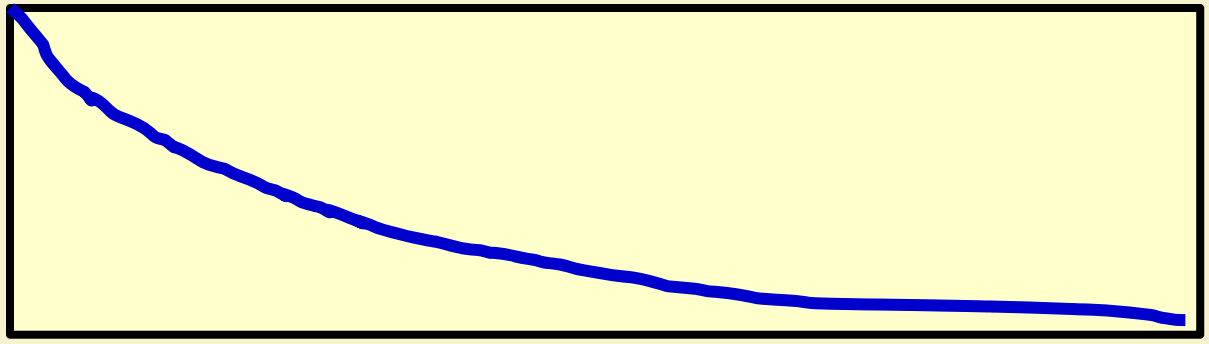
- Testing is often viewed as not very challenging --- less preferred by novices, but:
  - Over the years testing has taken a center stage in all types of software development.
  - **"Monkey testing is passe" --- Large number of innovations have taken place in testing area --- requiring tester to have good knowledge of test techniques.**
  - Challenges of test automation

Testing Activities Now Spread Over Entire Life Cycle



Test How Long?

- **One way:**



- **Another way:**
  - Seed bugs… run test cases
  - See if all (or most) are getting detected

Verification versus Validation

- Verification is the process of determining:
  - Whether output of one phase of development conforms to its previous phase.
- Validation is the process of determining:
  - Whether a fully developed system conforms to its SRS document..


Thank you